I don’t want to make it seem like using a reverse proxy is all sunshine and roses. There are some downsides:

- *Complexity*—One of the biggest complaints is how complex reverse proxies can be. If you’re managing the proxy yourself (as opposed to relying on a PaaS implementation), there can be lots of proxy-specific pitfalls to look out for.
- *Inter-process communication*—Most reverse proxies require two processes: a reverse proxy and your web app. Communicating between the two is often slower than if you directly exposed your web app to requests from the internet.
- *Restricted features*—Not all reverse proxies support all the same features as an ASP.NET Core app. For example, Kestrel supports HTTP/2, but if your reverse proxy doesn’t, you won’t see the benefits.

Whether you choose to use a reverse proxy or not, when the time comes to host your app, you can’t copy your code files directly to the server. First, you need to *publish* your ASP.NET Core app to optimize it for production. In section 27.1.1 we’ll look at building an ASP.NET Core app so that it can be run on your development machine, compared with publishing it so that it can be run on a server.

### *27.1.1 Running vs. publishing an ASP.NET Core app*

One of the key changes in ASP.NET Core from previous versions of ASP.NET is making it easy to build apps using your favorite code editors and integrated development environments (IDEs). Previously, Visual Studio was required for ASP.NET development, but with the .NET command-line